

Matthew Bernardino
Professor Anshul
ECPE 136
5 February 2026

Lab Assignment 2- Carry Look Ahead Adder

Objective:

The objective of this lab assignment is to learn and simulate a 16-bit carry look ahead adder. A carry look ahead adder is used to compute faster the addition of carry signals in parallel instead of waiting for each carry to start the next computation. In the ripple carry adder we explored last assignment, each adder waited for the previous adder to compute the carry which takes more time. Compared to the carry look ahead adder which predicts the carries using generate and propagate signals. Within this lab we create 5 modules that consist of partial full adder, carry look ahead logic, 4 bit carry look ahead, 16bit carry look ahead, and then a test bench to iterate through values to confirm outputs and inputs.

Procedures:

1.0 - Review the Carry Look Ahead Adder architecture and theory:

- Given the lab assignment provided diagrams and information to understand how the carry look ahead adder works on both an equation and circuit. Understanding the concepts of generate and propagate are important to reducing carry propagation delay compared to a ripple carry adder.

2. Design of Partial Full Adder

- A partial full adder was implemented in SystemVerilog. The generate is represented as ($g = a \& b$), propagated as ($p = a \wedge b$), and sum ($s = a \wedge b \wedge \text{carry_in}$) defining the signals. The generate is the carry that is known and propagation is a possible carry.

3. Design Carry Look Ahead Logic

- A carry look ahead module was created to calculate all the carry outputs simultaneously. Since all computations of generate and propagations are independent of each other, a 4 bit carry look ahead block does not have to wait on another partial full adder's carry, making the calculation of carries faster.

4. 16-bit Carry Look Ahead with Hierarchy

- Using four 4-bit carry look ahead adders we are able to instantiate them to create a full 16 bit carry look ahead adder. An additional carry look ahead logic is added to compute carry signals from all four 4 bit carry look ahead blocks and using the group generate and propagate outputs, making the parallel calculation of the carries between blocks instead of waiting.

5. Test Bench Creation

- Making a test bench to test various inputs to verify carrying is important for this assignment. Within the testbench I set up a guaranteed carry out example and iterated from 0 for a and b to add and verify carry in or carry out along with the sum output.

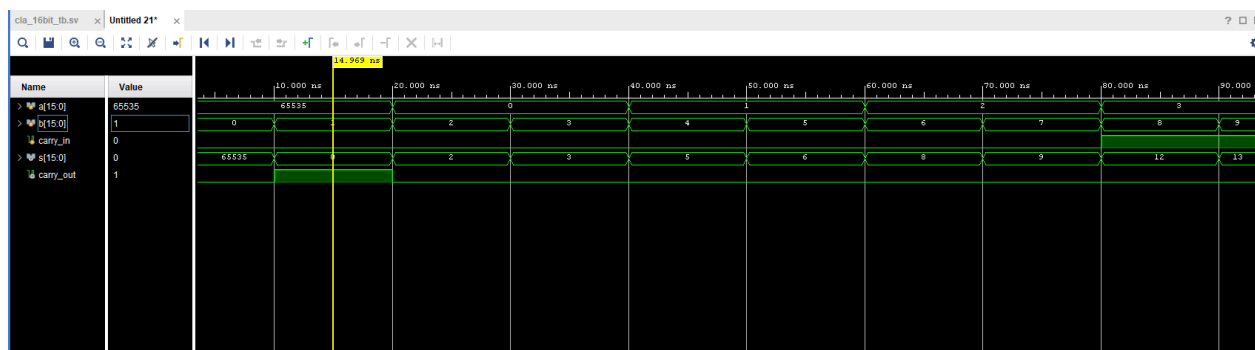
6 Simulation Waveform and RTL Schematic

- Using the testbench to verify multiple inputs of a and b for carry and sum output verification. A waveform was generated to represent the same inputs with their outputs and carries. With this we were able to analyze the addition through time as each input a and b changes updating the carries and output. Along with this a RTL schematic was generated to visualise the circuit and how it looks for a 16 bit carry look ahead adder.

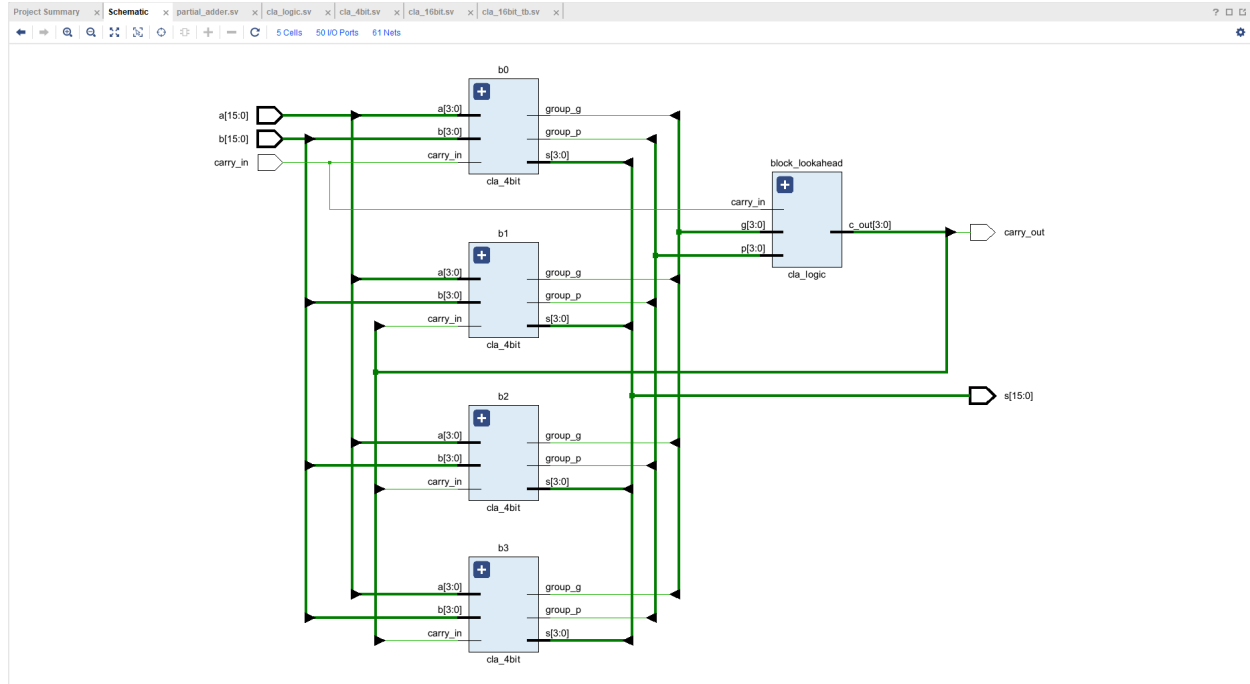
Observations:

- With test benching the 16-bit carry look ahead adder, I observed how the sum and carry outputs updated with a set time (ns) based on tested inputs. A specific input that was tested to verify carry out was the use of adding 65,535 with any other number. Since 65,535 is the max 16-bit output there is a guaranteed carry out with any number greater than 0. Another observation was iterating from both inputs a and b starting at 0. Making input b change value faster than a allows us to see the addition of multiple inputs and not just changing a and b at the same time. With this the waveform showed the input of a and its adding to different inputs b and updating the sum and carries. I got to see specifically at input a = 1 and input b = 4, sum = 5, when Carry in is also working properly when I noticed input a = 3 and input b = 8, cin = 1, sum = 12, this is the correct output as the carry adds an additional 1.

Results:



16-bit Carry Look Ahead Adder Waveform



16-bit Carry Look Ahead Adder RTL Schematic

Conclusion:

While working on this lab assignment I got to further my skills in learning how to use AMD VIVADO as a simulator and code compiler. I got to learn about the carry look ahead adder in which it allows us to add numbers even faster together. We explore parallelism in the fact that the carry look ahead adder allows for the calculations to be done simultaneously. The 16-bit look ahead adder was a success when test benching and understanding how the carry look ahead adds to the calculations. When looking at the waveform I noticed that the inputs a and b adding could be a base number but then the carry in would add an additional 1 to it making the sum greater than just what a and b add to. This was interesting to learn about the logic and how to understand the cases as well for when carried in will affect or not affect the sum. Analyzing edge cases like the maximum 16 bit number 65,535 was interesting to see the carryout working as it should. This lab let me get more comfortable with instantiations and making multiple files and testbenching to verify the output.

Appendix

```
// Matthew Bernardino
// ECPE 136
// Professor Anshul
// Description: Partial full adder - used to calculate if generate (carry produced) and propagate
(incoming carry) for carry

module partial_fulladder(
    // setup variables to be used for partial adder
    input logic a, b, carry_in, // 3 inputs
    output logic g, p, s // 3 outputs, generate, propagate, sum
);

    assign g = a & b;    // generate function (a carry bit is generated), a carry out is created no matter
    what carry in

    assign p = a ^ b;    // propagate function (will a carry bit pass through), this bit will pass the
    carry-in through to carry-out if one comes in

    assign s = a ^ b ^ carry_in; // sum function (sum bit for position)

endmodule
```

partial_adder.sv

```

// Matthew Bernardino
// ECPE 136
// Professor Anshul
// Description: cla logic - used to calculate all carries simultaneously using generate and propagate
signals and carry in

module cla_logic(
    // setup our 4 bit inputs and carry in
    input logic [3:0] g, // both propagate and generate are 4 bits
    input logic [3:0] p,
    input logic carry_in, // 1 or 0 for carry or no carry

    // setup our 4 bit outputs
    output logic [3:0] c_out,
    output logic group_g, // if generated carry from 4 bit
    output logic group_p // if propagated carry from 4 bit
);

    // we are showing all possible ways a carry can happen with each equation.

    assign c_out[0] = g[0] | (p[0] & carry_in); // carry into bit 1: generate at bit 0 or propagate from
carry_in

    assign c_out[1] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & carry_in); //carry into bit 2: generate at bit 1 or
generate at g[0] and p[1] or propagate all the way with carry in

    assign c_out[2] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & carry_in); //carry
into bit 3: generate at bit 2 or generate at g[0] and p[1] or propagate all the way with carry in

    assign c_out[3] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0]) | (p[3] &
p[2] & p[1] & p[0] & carry_in); // generate at bit 4 or propagate from previous

    assign group_p = p[3] & p[2] & p[1] & p[0]; // 4 bit might carry in if all 4 bits propagate, if not then
carry stops

    assign group_g = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0]); //
determines if entire 4 bit block generates a carry output, or if propagates with earlier generate to make
carry output without carry in needed

endmodule

```

cla_logic.sv

```
// Matthew Bernardino
// ECPE 136
// Professor Anshul
// Description: cla 4bit - instantiate 4 partial full adders and connect to carry look ahead logic setup in
cla_logic.sv
// Each partial full adder with compute the generate and propogate signals
// all generate and propogate go into the cla_logic for all 4 carries. this is done in parallel making it
faster, a final sum is created with the groups
```

```
module cla_4bit(
    // setup 4 bit input variables a/b
    input logic [3:0] a,
    input logic [3:0] b,
    input logic carry_in,

    // setup our output variables that connect to previous scripts
    output logic [3:0] s, // sum 4 bit output
    output logic group_g, //group_g 4 bit from cla logic
    output logic group_p, //group_p 4 bit from cla logic
    output logic carry_out
);

// create variables to connect to cla logic from cla_logic.sv
logic [3:0] generate_s; // generate array for g[0] - g[3]
logic [3:0] propogate_s; // propagate array for p[0] - p[3]
logic [3:0] local_carry; // array of carries that go from partial adder to partial adder

// instantiate 4 partial full adders that calculate each bit position 0000 from partial_adder.sv
//a and b computation that determines g and p for each partial adder to create a 4 bit number in
parallel
    partial_fulladder bit0(
        .a(a[0]),
        .b(b[0]),
        .carry_in(carry_in), // first carry
        .g(generate_s[0]),
        .p(propogate_s[0]),
        .s(s[0])
    );
    partial_fulladder bit1(
        .a(a[1]),
        .b(b[1]),
        .carry_in(local_carry[0]), // carry from cla_logic
        .g(generate_s[1]),
        .p(propogate_s[1]),
        .s(s[1])
    );

    partial_fulladder bit2(
        .a(a[2]),
        .b(b[2]),
```

```

        .carry_in(local_carry[1]), // carry from cla_logic
        .g(generate_s[2]),
        .p(propogate_s[2]),
        .s(s[2])
    );

    partial_fulladder bit3(
        .a(a[3]),
        .b(b[3]),
        .carry_in(local_carry[2]), // carry from cla_logic
        .g(generate_s[3]),
        .p(propogate_s[3]),
        .s(s[3])
    );

    // need to instantiate the cla logic from cla_logic.sv
    // take all g and p and calculate all carries simultaneously

    cla_logic lookahead_cla(
        .g(generate_s),
        .p(propogate_s),
        .carry_in(carry_in),
        .c_out(local_carry),
        .group_g(group_g),
        .group_p(group_p)
    );

    assign carry_out = local_carry[3]; // final output is the carry out of bit 3, gets used as the next carry
    input for the next 4 bit block when making 16 bit total cla.

endmodule

```

cla_4bit.sv

```
// Matthew Bernardino
// ECPE 136
// Professor Anshul
// Description: cla 16bit - use 4 cla 4 bit partial full adder blocks to make 16 bit cla adder.
// Each 4 bit block will do its own simultaneous look ahead with the 4 bit cla to comput a 16 bit cla
adder.
// The blocks will look ahead on their own bits and then a final look ahead looks over all the 4 bit
blocks
```

```
module cla_16bit(
// need 16 bit input variables for all 16 input bits for a and b
    input logic [15:0] a,
    input logic [15:0] b,
    input logic carry_in, // 16 bit carry in

    output logic [15:0] s, // total 16 bit sum output
    output logic carry_out // 16 bit carry out if any
);

// need internal logic for connecting the 4 bit partial full adder blocks

    logic [3:0] group_gblock; // generate 4 bit group block
    logic [3:0] group_pblock; // propagate 4 bit group block
    logic [3:0] local_carry_block; //carries from the 4 bit cla partial full adder blocks

    cla_4bit b0( // first 4 bits 0 - 3
        .a(a[3:0]), //4 bit array for output of partial full adder being 0000
        .b(b[3:0]),
        .carry_in(carry_in), // external carry in
        .s(s[3:0]), // 4 bit sum 0000
        .carry_out(), // this gets ignored because of lookahead
        .group_g(group_gblock[0]), // handles extra carry
        .group_p(group_pblock[0])
    );

    cla_4bit b1( // next 4 bits 4 - 7
        .a(a[7:4]), //4 bit array for output of partial full adder being 0000
        .b(b[7:4]),
        .carry_in(local_carry_block[0]), // lookahead logic
        .s(s[7:4]), // 4 bit sum 0000
        .carry_out(), // this gets ignored because of lookahead
        .group_g(group_gblock[1]), // handles extra carry
        .group_p(group_pblock[1])
    );

    cla_4bit b2( // next 4 bits 8 - 11
        .a(a[11:8]), //4 bit array for output of partial full adder being 0000
        .b(b[11:8]),
        .carry_in(local_carry_block[1]),
        .s(s[11:8]), // 4 bit sum 0000
```



```

        .carry_out(), // this gets ignored because of lookahead
        .group_g(group_gblock[2]), // handles extra carry
        .group_p(group_pblock[2])
    );

    cla_4bit b3( // next 4 bits 12 - 15
        .a(a[15:12]), //4 bit array for output of partial full adder being 0000
        .b(b[15:12]),
        .carry_in(local_carry_block[2]),
        .s(s[15:12]), // 4 bit sum 0000
        .carry_out(), // this gets ignored because of lookahead, no need to wait on other block to feed
output
        .group_g(group_gblock[3]), // handles extra carry
        .group_p(group_pblock[3])
    );

    // need to instantiate to keep the lookahead continuing to prevent bottleneck
    // we dont want the blocks waiting on one another, by converting each block to a single bit, the
lookahead can continue
    // takes the whole group_gblock / group_pblock 4 bit block and carry in and computes
local_carry_block(4 bits) in parallel
    cla_logic block_lookahead(
        .g(group_gblock),
        .p(group_pblock),
        .carry_in(carry_in),
        .c_out(local_carry_block),
        .group_g(), // not needed as no larger adder than 16, no need to propagate / generate
        .group_p()
    );

    // final carry outgoing into another block if there was one.
    assign carry_out = local_carry_block[3];

endmodule

```

cla_16bit.sv

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 02/05/2026 03:47:12 PM
// Design Name:
// Module Name: cla_16bit_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
/////////////////////////////////////////////////////////////////
//Matthew Bernardino
//Test bench for 16 bit carry look ahead adder

module cla_16bit_tb();
    reg [15:0] a; // 16 bit variables a and b for adding
    reg [15:0] b;
    reg carry_in; // carry in variable to account for any

    wire [15:0] s; // total result of 16 bit output
    wire carry_out; // if a carry out exists then account for it

    //instantiate the design module and connect the variable declared above
    cla_16bit cla_16bit_inst(.a(a), .b(b), .carry_in(carry_in), .s(s), .carry_out(carry_out));

    initial begin

        a = 16'hFFFF; // want to simulate carry out, so 65535 plus any num>0 is cout=1
        b = 16'h0000;
        carry_in = 1'b0;

        #200

        a = 16'b0; // we set both a and b to 0 to start incrementing with time
        b = 16'b0;
        carry_in = 1'b0; // only a 1 bit extra carry of there is

        #800 $finish; //200 ns time

    end

```

```
always #20 a = a + 1; // increment 0 - 15 for testing
always #10 b = b + 1; // increment 0 - 15 for testing, do so faster to add the same value to a for
multiple cases
always #80 carry_in = ~carry_in; // carry in setup

always @(a, b, carry_in, s, carry_out)
    $monitor("at time=%t ns, a=%0d, b=%0d, cin=%1b, sum=%0d, cout=%1b", $time, a, b, carry_in,
s, carry_out);

endmodule
```

cla_16bit_tb.sv